NoSQL, Elasticsearch & Solr

40 Interview Questions & Answers

In Simple, Easy Language — From Basics to Advanced

✓ NoSQL Core Concepts — Types, CAP Theorem, Consistency
✓ MongoDB — Documents, BSON, Aggregation, Sharding, Replication
✓ Redis — Data Structures, Caching, Pub/Sub, Persistence
✓ Elasticsearch — Index, Shards, Inverted Index, ELK Stack, Queries
✓ Apache Solr — Architecture, SolrCloud, Faceting
✓ Elasticsearch vs Solr — Head-to-head comparison

Colour Legend

Section	Question Colour	Coverage
NoSQL Core Concepts (Q1–Q10)	Blue	CAP theorem, types, BASE, scaling, sharding
MongoDB (Q11–Q20)	Brown/Orange	Documents, CRUD, aggregation, indexing, replication
Redis (Q21–Q27)	Red	Data structures, TTL, persistence, Pub/Sub, caching
Elasticsearch (Q28–Q35)	Teal	Index, shards, inverted index, queries, ELK stack
Apache Solr & Comparison (Q36–Q40)	Purple	SolrCloud, faceting, Solr vs Elasticsearch

Tip: All answers start with a simple, everyday explanation first — then go deeper with technical details and code examples. This way you can give a short answer if the interviewer wants a quick response, or extend it when they ask for more detail.

Table of Contents

1. Section 1: NoSQL Core Concepts (Q1–Q10)
2. Section 2: MongoDB — Document Database (Q11–Q20)
3. Section 3: Redis — Key-Value Store & Cache (Q21–Q27)
4. Section 4: Elasticsearch — Search Engine (Q28–Q35)
5. Section 5: Apache Solr & ES vs Solr Comparison (Q36–Q40)
6. Quick Reference: NoSQL Comparison Table

Section 1: NoSQL Core Concepts (Q1–Q10)

These questions apply to ALL NoSQL databases. They test whether you understand WHY NoSQL exists, what problems it solves, and the theory behind distributed databases.

Q1: What is NoSQL? Why was it created? How is it different from SQL?

Answer: NoSQL means 'Not Only SQL'. It was created to handle problems that traditional relational databases struggle with: storing huge volumes of unstructured or semi-structured data, scaling horizontally across many servers, and handling flexible or rapidly changing data structures. SQL databases store data in rigid, structured tables with fixed columns. NoSQL databases store data in flexible formats — documents (JSON), key-value pairs, wide columns, or graph nodes. The trade-off: NoSQL gains flexibility and scalability but often relaxes strict ACID consistency rules.

Q2: What are the four types of NoSQL databases? Explain each with real-world examples.

Answer: Document Store: stores data as JSON/BSON documents. Each document can have different fields. Example: MongoDB, CouchDB. Used for: e-commerce products, user profiles, content management. Key-Value Store: stores data as simple key → value pairs — like a dictionary. Very fast. Example: Redis, DynamoDB. Used for: caching, sessions, leaderboards, real-time counters. Column-Family (Wide-Column) Store: stores data in columns grouped by rows, but each row can have different columns. Example: Apache Cassandra, HBase. Used for: time-series data, IoT sensor data, analytics at massive scale. Graph Database: stores data as nodes (entities) and edges (relationships). Example: Neo4j. Used for: social networks, fraud detection, recommendation engines.

Q3: What is the CAP Theorem? Why is it important for NoSQL?

Answer: The CAP Theorem (Brewer's Theorem) states that a distributed database can only guarantee TWO out of three properties at any given time. Consistency (C): every node returns the latest, most up-to-date data. Availability (A): every request always gets a response (even if data might be slightly stale). Partition Tolerance (P): the system keeps working even if network partitions (communication failures between nodes) occur. Since network partitions ALWAYS happen in real distributed systems, every NoSQL database must choose between C and A. MongoDB and HBase choose CP (consistent but may reject requests during network issues). Cassandra and CouchDB choose AP (always available but data might be slightly stale).

```
CAP Triangle — pick any two:

         Consistency (C)
              /\
             /  \
            / CP \  <- MongoDB, HBase
           /      \
          /  CA    \  <- Traditional RDBMS (single node)
         /----------\
Availability (A)   Partition Tolerance (P)
       AP -> Cassandra, CouchDB, DynamoDB
```

Q4: What is BASE and how is it different from ACID?

Answer: ACID (SQL databases): Atomicity, Consistency, Isolation, Durability — strict, strong guarantees. Every transaction is perfect or it doesn't happen. BASE (NoSQL databases): Basically Available, Soft state, Eventually Consistent. Basically Available: the system is almost always responding. Soft state: data values may change over time even without input (as replicas sync). Eventually Consistent: all replicas will converge to the same value eventually — just not instantly. Analogy: ACID is like a bank transfer that is either 100% complete or 100% undone. BASE is like DNS — if you update a domain, it takes a few hours to propagate everywhere, but eventually everyone sees the same record.

Q5: What is horizontal vs vertical scaling? Why do NoSQL databases prefer horizontal scaling?

Answer: Vertical Scaling (Scale Up): add more CPU, RAM, or disk to one server. Has a physical ceiling — you can only make one machine so powerful, and it's expensive. Horizontal Scaling (Scale Out): add more servers to the cluster. Theoretically unlimited and cheaper. NoSQL databases are designed from the ground up for horizontal scaling — you add nodes and the database automatically redistributes data. SQL databases are harder to scale horizontally because of joins, transactions, and the need for data consistency across nodes.

```
Vertical (Scale Up):     Horizontal (Scale Out):

  [Big Server]          [Small] [Small] [Small]
  More CPU/RAM     vs   [Small] [Small] [Small]
  Has ceiling           Add nodes = more capacity
  Expensive             Cheaper, unlimited
```

Q6: What is sharding in NoSQL? How does it work?

Answer: Sharding is splitting data across multiple servers (called shards) so no single server holds all the data. Each shard stores a subset of the total data. This allows the database to store more data than any single machine can hold and serve more requests in parallel. Shard Key: the column or field used to decide which shard a document goes to. Choosing a good shard key is critical — a bad one causes 'hot spots' where one shard gets most requests. Types: Range sharding (divide by value range, e.g., dates), Hash sharding (hash the key, even distribution), Directory sharding (lookup table maps keys to shards).

```
Without sharding:    With sharding (3 shards):

 [Server 1]          Shard 1: users A-H
 ALL 10M users  -->  Shard 2: users I-P
 Overloaded!         Shard 3: users Q-Z
                     Each shard: ~3.3M users
```

Q7: What is replication in NoSQL databases?

Answer: Replication means keeping copies (replicas) of the same data on multiple servers. This provides: High Availability — if one server fails, another replica takes over. Read Scalability — read requests can be distributed across replicas. Disaster Recovery — data is safe even if a data center fails. Primary-Secondary Replication: one primary node accepts all writes, secondary nodes copy data from primary and serve reads. Peer-to-Peer Replication: all nodes are equal and accept both reads and writes (used by Cassandra). The trade-off: more replicas = more safety but slower writes (must write to multiple places).

Q8: What is eventual consistency? When is it acceptable?

Answer: Eventual consistency means that all replicas of the data will eventually have the same value — but there may be a brief window where different replicas show different values. Example: you post a photo on Instagram. For a few seconds, some users might not see it yet (it's being replicated to other servers). Eventually everyone sees it. This is acceptable for: social media feeds, product recommendations, shopping carts (slight delay is fine), analytics data. It is NOT acceptable for: bank balances, inventory counts (selling the last item twice), medical records — anything where incorrect data can cause serious harm.

Q9: When should you choose NoSQL over SQL? Give real scenarios.

Answer: Choose NoSQL when: 1) Your data is unstructured or schema changes frequently (e.g., product catalog where each product has different attributes). 2) You need to store and query massive volumes of data beyond what one SQL server can handle. 3) You need high write throughput (millions of inserts per second — e.g., IoT sensor data, logs). 4) You need horizontal scalability cheaply. 5) Your access patterns are simple and known in advance (no complex ad-hoc joins). Choose SQL when: relationships between data are complex, you need strong ACID transactions, your team knows SQL well, your data model is stable and well-defined.

Q10: What is the difference between embedding and referencing in NoSQL data modelling?

Answer: Embedding: store related data inside the same document. Like storing a user's address inside the user document. Fast reads (one query), but documents can get large and updating nested data is complex. Good when: data is always read together, relationships are one-to-few. Referencing: store a reference (ID) to another document, similar to a foreign key in SQL. Smaller documents, easier to update referenced data separately. Requires multiple queries. Good when: data is large, reused across many documents, or relationships are one-to-many.

```
// Embedding (address inside user document)
{
  '_id': '123',
  'name': 'Alice',
  'address': {           // embedded
    'street': '12 MG Road',
    'city': 'Bangalore'
  }
}

// Referencing (order references user by ID)
{
  '_id': 'order_456',
  'user_id': '123',      // reference
  'total': 1500
}
```

Section 2: MongoDB — Document Database (Q11–Q20)

MongoDB is the most popular NoSQL database. It stores data as JSON-like documents in collections. These are the most commonly asked MongoDB interview questions.

Q11: What is MongoDB? What are its key features?

Answer: MongoDB is a free, open-source document-oriented NoSQL database. Instead of tables and rows, it uses collections and documents. Each document is a JSON-like structure (stored internally as BSON — Binary JSON) that can have any fields. Key features: flexible schema (no fixed columns), horizontally scalable via sharding, high write performance, built-in replication (Replica Sets), powerful aggregation framework, rich query language, support for ACID transactions (since MongoDB 4.0 for multi-document), and geospatial queries. Used by: eBay, Uber, Forbes, Adobe, and many more.

```
// A MongoDB document (vs a SQL row)
{
  '_id': ObjectId('64a1f2...'),
  'name': 'Alice Kumar',
  'email': 'alice@example.com',
  'skills': ['Java', 'Python', 'MongoDB'],  // array
  'address': {                               // nested object
    'city': 'Bangalore',
    'pincode': '560001'
  },
  'createdAt': ISODate('2024-06-01')
}
```

Q12: What is BSON and how is it different from JSON?

Answer: JSON (JavaScript Object Notation) is the text format you write and read. BSON (Binary JSON) is how MongoDB actually stores data on disk — it's a binary-encoded version of JSON. Why BSON? It's faster to parse, supports more data types than JSON (like Date, ObjectId, Binary data, 32-bit and 64-bit integers), and is more space-efficient for certain types. When you insert a document in MongoDB, it is automatically converted from JSON to BSON internally. You work with JSON; MongoDB stores BSON.

Q13: What is the basic CRUD in MongoDB? Show examples.

Answer: CRUD stands for Create, Read, Update, Delete — the four fundamental database operations. In MongoDB these are performed with insertOne/insertMany, find/findOne, updateOne/updateMany, and deleteOne/deleteMany.

```
// CREATE — insert a document
db.users.insertOne({name: 'Alice', age: 28, city: 'Mumbai'});
db.users.insertMany([{name:'Bob'}, {name:'Carol'}]);

// READ — find documents
db.users.find({city: 'Mumbai'});          // all Mumbai users
db.users.findOne({name: 'Alice'});         // first match
db.users.find({age: {$gt: 25}});           // age > 25

// UPDATE — modify documents
db.users.updateOne(
  {name: 'Alice'},
  {$set: {age: 29}}   // $set updates only this field
);

// DELETE
db.users.deleteOne({name: 'Alice'});
db.users.deleteMany({age: {$lt: 18}});    // delete all under 18
```

Q14: What are MongoDB query operators? List the most important ones.

Answer: MongoDB uses operators (starting with $) for filtering, updating, and aggregating. Comparison operators: $eq (equal), $ne (not equal), $gt (greater than), $gte (>=), $lt (less than), $lte (<=), $in (in array), $nin (not in array). Logical operators: $and, $or, $not, $nor. Array operators: $all (array contains all values), $size (array length), $elemMatch (match element in array). Update operators: $set (set field), $unset (remove field), $inc (increment), $push (add to array), $pull (remove from array).

```
// Comparison: salary between 50000 and 100000
db.employees.find({salary: {$gte: 50000, $lte: 100000}});

// Logical: city is Bangalore OR Mumbai
db.users.find({$or: [{city:'Bangalore'}, {city:'Mumbai'}]});

// Array: find users with skill 'Java'
db.users.find({skills: {$in: ['Java']}});

// Update: increment likes by 1
db.posts.updateOne({_id: id}, {$inc: {likes: 1}});
```

Q15: What is the MongoDB Aggregation Pipeline?

Answer: The Aggregation Pipeline is MongoDB's way of processing and transforming data in multiple stages — similar to SQL's GROUP BY, JOIN, and SELECT combined. Data flows through stages like water through pipes, each stage transforming it. Common stages: $match — filter documents (like WHERE). $group — group and aggregate (like GROUP BY + COUNT/SUM). $sort — sort results (like ORDER BY). $project — shape the output (like SELECT specific columns). $lookup — join with another collection (like LEFT JOIN). $limit, $skip — pagination. $unwind — expand an array into separate documents.

```
// Find total sales per city, sorted by sales descending
db.orders.aggregate([
  { $match: { status: 'completed' } },      // filter first
  { $group: {
      _id: '$city',
      totalSales: { $sum: '$amount' },
      orderCount: { $sum: 1 }
  }},
  { $sort: { totalSales: -1 } },             // highest first
  { $limit: 10 }                             // top 10 cities
]);
```

Q16: What is indexing in MongoDB? What types of indexes are available?

Answer: MongoDB indexes speed up queries — without an index, MongoDB does a 'collection scan' (reads every document). Types of indexes: Single Field Index — on one field (most common). Compound Index — on multiple fields. Text Index — for full-text search within string fields. Geospatial Index (2dsphere/2d) — for location-based queries. TTL Index (Time-To-Live) — automatically deletes documents after a specified time (great for sessions, logs). Hashed Index — for hash-based sharding. Multikey Index — automatically created for array fields.

```
// Single field index (on email for fast login lookup)
db.users.createIndex({email: 1});  // 1 = ascending

// Compound index
db.orders.createIndex({customer_id: 1, created_at: -1});

// TTL index — document expires after 3600 seconds
db.sessions.createIndex({createdAt: 1}, {expireAfterSeconds: 3600});

// Text index for full-text search
db.articles.createIndex({title: 'text', body: 'text'});
db.articles.find({$text: {$search: 'mongodb performance'}});

// See all indexes on a collection
db.users.getIndexes();
```

Q17: What is a Replica Set in MongoDB?

Answer: A Replica Set is a group of MongoDB servers that maintain the same data. It provides high availability and data redundancy. Structure: one Primary node (accepts all writes and reads) and one or more Secondary nodes (constantly copy data from primary and can serve reads). Arbiter (optional): a node that votes in elections but does not store data. Failover: if the primary goes down, the secondaries hold an automatic election and one becomes the new primary — typically takes 10-30 seconds. Minimum recommended: 3 nodes (1 primary + 2 secondaries) so there's always a majority for elections.

```
// Replica Set topology
Primary  ---replicates--->  Secondary 1
         ---replicates--->  Secondary 2

// If Primary fails:
// Secondary 1 and Secondary 2 hold election
// One becomes the new Primary automatically

// Read from secondary (lower latency for reads)
db.getMongo().setReadPref('secondaryPreferred')
```

Q18: How does MongoDB Sharding work?

Answer: MongoDB sharding distributes data across multiple servers (shards). Three components: Shards — the servers that actually store data partitions. Config Servers — store metadata about where data lives (which shard holds which range). Mongos (Router) — the query router that directs requests to the right shard. Applications connect to mongos, not directly to shards. Mongos uses config servers to find which shard has the data. Shard Key: the field used to decide how data is split. Choose carefully — a shard key with low cardinality (e.g., boolean field) is a very bad choice. Good shard keys: user ID, timestamp, order ID.

```
Shard Key: user_id

App → Mongos (Router)
         ↓
   Config Servers (metadata: which shard holds what range)
         ↓              ↓              ↓
   [Shard 1]       [Shard 2]       [Shard 3]
  user_id 1-33%   user_id 33-66%  user_id 66-100%
```

Q19: Does MongoDB support transactions? What were the limitations?

Answer: Yes, MongoDB supports ACID transactions — but the story evolved over versions. Before MongoDB 4.0: only single-document operations were atomic. Multi-document operations were NOT atomic. MongoDB 4.0 (2018): added multi-document ACID transactions for Replica Sets. MongoDB 4.2: extended transactions to sharded clusters. The caveat: multi-document transactions in MongoDB are more expensive (performance overhead) than in SQL databases. Best practice: design your schema to avoid needing transactions where possible (embed related data in one document). Use transactions only when truly required for multi-collection consistency.

```
// Multi-document transaction (MongoDB 4.0+)
const session = client.startSession();
session.startTransaction();
try {
  db.accounts.updateOne({_id: 'A'}, {$inc: {balance: -500}}, {session});
  db.accounts.updateOne({_id: 'B'}, {$inc: {balance: +500}}, {session});
  await session.commitTransaction();
} catch (e) {
  await session.abortTransaction();  // rollback both
} finally {
  session.endSession();
}
```

Q20: What is $lookup in MongoDB and how does it compare to SQL JOIN?

Answer: $lookup is the MongoDB aggregation stage for joining two collections — similar to a SQL LEFT OUTER JOIN. It adds an array of matching documents from another collection to each document in the pipeline. Important caveat: MongoDB is designed for embedding related data, not joining. $lookup is powerful but should be used sparingly — if you find yourself doing $lookup on every query, reconsider your schema design. For simple embedding patterns, $lookup is unnecessary.

```
// $lookup: join orders with their customer details
db.orders.aggregate([
  {
    $lookup: {
      from: 'customers',         // join with this collection
      localField: 'customer_id', // field in orders
      foreignField: '_id',       // field in customers
      as: 'customer_info'        // output array field name
    }
  },
  { $unwind: '$customer_info' }  // flatten the array to one object
]);

// SQL equivalent:
// SELECT o.*, c.* FROM orders o
// LEFT JOIN customers c ON o.customer_id = c.id
```

Section 3: Redis — Key-Value Store & Cache (Q21–Q27)

Redis (Remote Dictionary Server) is an in-memory data structure store used as a cache, message broker, and database. It is blazing fast and one of the most widely used tools in modern application stacks.

Q21: What is Redis and what makes it so fast?

Answer: Redis is an in-memory, open-source key-value store. 'In-memory' is the key — all data lives in RAM, not on disk. RAM access is thousands of times faster than disk access, which is why Redis can handle millions of operations per second. Beyond simple key-value pairs, Redis supports rich data structures. It is single-threaded (in its command processing), which eliminates locking overhead. Use cases: caching (most common), session storage, rate limiting, leaderboards, pub/sub messaging, real-time analytics, job queues. Popular companies using Redis: Twitter (for timelines), GitHub (for queues), Pinterest (for counting).

Q22: What data structures does Redis support? Give a use case for each.

Answer: Redis is not just a cache — it's a data structure server. String: simplest type, stores text or numbers. Use: caching HTML, counters (page views). List: ordered list, like a linked list. Use: message queues, activity feeds, recent items. Set: unordered collection of unique values. Use: unique visitors, tags, social graph connections. Sorted Set (ZSet): set with a score for ranking. Use: leaderboards, priority queues, rate limiters. Hash: map of field-value pairs (like a mini-document). Use: storing user profiles, object properties. Bitmap: bit-level operations. Use: tracking daily active users. HyperLogLog: probabilistic counting with tiny memory. Use: approximate unique count at massive scale.

```
// String — cache a user's profile
SET user:123 '{"name":"Alice","city":"Bangalore"}'
GET user:123

// List — message queue (push left, pop right = FIFO queue)
LPUSH notifications 'Order shipped'
RPOP notifications

// Sorted Set — leaderboard (score = points)
ZADD leaderboard 1500 'Alice'
ZADD leaderboard 2000 'Bob'
ZREVRANGE leaderboard 0 9 WITHSCORES  // top 10

// Hash — store user fields separately
HSET user:456 name 'Bob' email 'bob@ex.com' age 30
HGET user:456 name    // Bob
HGETALL user:456      // all fields
```

Q23: What is TTL (Time To Live) in Redis and why is it important?

Answer: TTL is the expiry time set on a key — Redis automatically deletes the key after the specified time. This is crucial for caching because: you don't want stale data to live forever, and RAM is finite — automatically expiring old entries prevents the cache from filling up. Set TTL with EXPIRE (seconds) or EXPIREAT (Unix timestamp) or inline with SET ... EX. Check TTL remaining with TTL key (returns -1 if no expiry, -2 if key doesn't exist).

```
// Set a key with 1 hour (3600 second) TTL
SET session:abc123 '{"user_id": 42}' EX 3600

// Set TTL on existing key
EXPIRE otp:9876543210 300  // OTP expires in 5 minutes

// Check remaining TTL
TTL session:abc123    // returns e.g. 3542 (seconds left)
```

```
// Remove expiry (make permanent)
PERSIST session:abc123

// SETNX — set only if key does NOT exist (distributed lock pattern)
SETNX lock:resource 'locked' EX 30
```

Q24: How does Redis persistence work? What are the options?

Answer: Since Redis is in-memory, data is lost if the server restarts — unless you enable persistence. Two options: RDB (Redis Database) Snapshots: Redis takes a snapshot of all data at configured intervals and saves it to an .rdb file on disk. Fast startup, compact file. Risk: you can lose data since the last snapshot. AOF (Append Only File): Redis logs every write operation. On restart, it replays the log to rebuild data. More durable (lose at most 1 second of data) but larger file and slower startup. Combined RDB + AOF: recommended for most production setups. Best durability. For pure cache use cases, disabling persistence (no-save mode) is fine — data loss is acceptable since it's just a cache.

```
# redis.conf — RDB snapshot every 60 seconds if 1000 keys changed
save 60 1000

# Enable AOF (append only file)
appendonly yes
appendfsync everysec   # sync to disk every 1 second (good balance)

# Combined (recommended for production)
save 3600 1
appendonly yes
```

Q25: What is Redis Pub/Sub? What is it used for?

Answer: Pub/Sub (Publish/Subscribe) is a messaging pattern in Redis where: Publishers send messages to a channel (without knowing who is listening). Subscribers listen to a channel (without knowing who is sending). Redis acts as the message broker in the middle. Use cases: real-time notifications (chat messages, alerts), broadcasting events to multiple services, live dashboards updating in real time. Important limitation: Redis Pub/Sub is fire-and-forget — if a subscriber is offline when a message is published, that message is lost permanently. For reliable messaging (guaranteed delivery), use Redis Streams instead.

```
// Terminal 1 — SUBSCRIBE (listening)
SUBSCRIBE notifications

// Terminal 2 — PUBLISH (sending)
PUBLISH notifications 'New order placed: #12345'

// Terminal 1 receives:
// 1) message
// 2) notifications
// 3) New order placed: #12345

// Pattern subscribe (listen to all 'order.*' channels)
PSUBSCRIBE order.*
```

Q26: What is Redis Cluster and how does it provide scalability?

Answer: A single Redis instance has limits — it runs on one machine, single-threaded, and is bound by that machine's RAM. Redis Cluster allows distributing data across multiple Redis nodes automatically. It uses hash slots — the keyspace is divided into 16,384 hash slots. Each master node in the cluster handles a range of hash slots. When you write a key, Redis hashes it to find which slot (and therefore which node) it belongs to. Each master node also has one or more replica nodes for failover. Redis Cluster provides both data partitioning (scale out) and high availability (replicas).

Q27: What is a cache eviction policy in Redis? Which should you choose?

Answer: When Redis runs out of memory, it needs to decide which keys to delete to make room. This is the eviction policy. Common policies: noeviction: refuse new writes and return an error (bad for

caches, okay for queues). allkeys-lru: evict the Least Recently Used key from ALL keys (most common for general caches). volatile-lru: evict LRU keys only from keys that have a TTL set. allkeys-random: remove a random key. volatile-ttl: evict the key with the shortest remaining TTL. allkeys-lfu (Redis 4.0+): evict the Least Frequently Used key. Best practice for caches: use allkeys-lru. This ensures the cache stays within memory limits and old/unused data gets removed first.

```
# Set in redis.conf or at runtime
maxmemory 4gb
maxmemory-policy allkeys-lru

# Check current memory usage
INFO memory

# Change policy at runtime (no restart needed)
CONFIG SET maxmemory-policy allkeys-lru
```

Section 4: Elasticsearch — Search Engine (Q28–Q35)

Elasticsearch is a distributed search and analytics engine built on Apache Lucene. It is the most popular choice for full-text search, log analytics, and the backbone of the ELK Stack.

Q28: What is Elasticsearch and what are its main use cases?

Answer: Elasticsearch is an open-source, distributed search and analytics engine built on top of Apache Lucene. It stores data as JSON documents and makes them searchable in near real-time (about 1 second delay between indexing and searching). It's accessible via a RESTful HTTP API. Main use cases: Full-text search (product search, website search, code search). Log and event data analysis (the 'E' in ELK/Elastic Stack). Application performance monitoring. Security analytics. E-commerce search with facets and filters. Geospatial search. Well-known users: GitHub (code search), Netflix (log analytics), Wikipedia (search), Uber (geospatial).

Q29: Explain the core concepts of Elasticsearch: Index, Document, Shard, Node, Cluster.

Answer: Cluster: a collection of one or more Elasticsearch nodes that work together. Identified by a cluster name. Node: a single Elasticsearch server instance. Each node stores data and participates in indexing and searching. Index: like a 'database' in SQL — a logical namespace for related documents. Example: 'products' index, 'logs' index. Shard: an index is split into shards — each shard is a complete Lucene index. Shards enable distributing an index across multiple nodes. Primary Shard: holds the actual data. Replica Shard: a copy of a primary shard for redundancy and read scalability. Document: the basic unit of data — a JSON object. Like a 'row' in SQL.

```
SQL analogy vs Elasticsearch:

SQL Database    → Elasticsearch Index
SQL Table       → (Index, in ES types are deprecated)
SQL Row/Record  → ES Document (JSON)
SQL Column      → ES Field
SQL Schema      → ES Mapping

Example: products index with 5 shards:
Shard 0 → Node 1 (primary)    Shard 0 → Node 2 (replica)
Shard 1 → Node 2 (primary)    Shard 1 → Node 3 (replica)
... and so on
```

Q30: What is an inverted index and why is it the heart of Elasticsearch?

Answer: An inverted index is the data structure that makes fast full-text search possible. In a normal index (forward index), you store: document → words it contains. An inverted index flips this: word → list of documents that contain it. When you search for 'laptop performance', Elasticsearch looks up 'laptop' and 'performance' in the inverted index, finds all documents containing each word, and intersects the results. This is why full-text search is milliseconds fast even with billions of documents — you never scan the documents themselves.

```
Documents:
  Doc 1: 'Buy laptop online'
  Doc 2: 'Laptop performance review'
  Doc 3: 'Best online performance tips'

Inverted Index:
  'buy'         → [Doc 1]
  'laptop'      → [Doc 1, Doc 2]
  'online'      → [Doc 1, Doc 3]
  'performance' → [Doc 2, Doc 3]
  'review'      → [Doc 2]

Search 'laptop performance' → Doc 2 (contains both)
```

Q31: What is the ELK Stack? What does each component do?

Answer: ELK Stack (now called Elastic Stack) is a set of three open-source tools that work together for log management and analytics. E — Elasticsearch: stores, indexes, and makes logs searchable. The heart of the stack. L — Logstash: collects logs from various sources (files, databases, message queues), parses and transforms them, then sends them to Elasticsearch. K — Kibana: the visualisation layer — provides a web UI to search logs, create dashboards, charts, and alerts on top of Elasticsearch. Modern addition — Beats: lightweight agents (Filebeat, Metricbeat) installed on servers to ship logs directly to Elasticsearch or Logstash, replacing heavy Logstash agents.

```
ELK/Elastic Stack Flow:

Servers/Apps
     ↓  (Filebeat/Beats — lightweight log shipper)
  Logstash  (collect → parse → transform)
     ↓
Elasticsearch  (store → index → search)
     ↓
  Kibana  (visualise → dashboards → alerts)

Common use: centralised log management
- All microservice logs in one searchable place
- Real-time error alerting
- Performance dashboards
```

Q32: What are the different types of queries in Elasticsearch?

Answer: Elasticsearch has two main categories of query context. Query Context: relevance matters — results are scored and ranked by how well they match. Used for full-text search. Filter Context: yes/no match, no scoring — used for exact values, ranges, dates. Faster because results are cached. Main query types: match — full-text search on a field (most common). term — exact value match (not analysed). terms — match multiple exact values. range — date or number range. bool — combine multiple queries with must, should, must_not, filter. multi_match — search across multiple fields. wildcard, prefix, fuzzy — pattern and approximate matching.

```
// Bool query: most powerful, combines conditions
GET /products/_search
{
  'query': {
    'bool': {
      'must': [
        {'match': {'description': 'wireless headphones'}}
      ],
      'filter': [
        {'range': {'price': {'gte': 500, 'lte': 5000}}},
        {'term': {'in_stock': true}}
      ],
      'must_not': [
        {'term': {'category': 'accessories'}}
      ]
    }
  }
}
```

Q33: What is mapping in Elasticsearch? What is dynamic mapping?

Answer: Mapping defines the schema for an index — it specifies the data types (text, keyword, integer, date, geo_point, etc.) for each field and how they should be indexed and analysed. Dynamic Mapping: if you don't define a mapping, Elasticsearch automatically infers field types when you first index a document. Strings become text (full-text) with a keyword sub-field. Numbers become integer or float. Dates become date. Explicit Mapping: you define the mapping yourself before inserting. Important distinction: text vs keyword. text fields are analysed (tokenised, lowercased) for full-text search. keyword fields are stored as-is for exact matching, sorting, and aggregations.

```
// Create an index with explicit mapping
PUT /products
{
```

```
  'mappings': {
    'properties': {
      'name':     {'type': 'text'},          // full-text search
      'brand':    {'type': 'keyword'},        // exact match, aggregations
      'price':    {'type': 'float'},
      'in_stock': {'type': 'boolean'},
      'created':  {'type': 'date'},
      'location': {'type': 'geo_point'}      // geospatial
    }
  }
}
```

Q34: What are aggregations in Elasticsearch?

Answer: Aggregations are Elasticsearch's powerful analytics engine — similar to SQL GROUP BY, COUNT, SUM, AVG but much more flexible and multi-level. Three main types: Metric Aggregations: calculate statistics on field values — avg, sum, min, max, count, percentiles. Bucket Aggregations: group documents into buckets — terms (group by field value), date_histogram (group by time), range (group by value range). Pipeline Aggregations: take the output of other aggregations as input — moving averages, cumulative sums. You can nest aggregations inside each other for powerful multi-level analysis (e.g., total revenue per category per month).

```
// Average price per brand, with total count
GET /products/_search
{
  'size': 0,
  'aggs': {
    'by_brand': {
      'terms': {'field': 'brand', 'size': 10},
      'aggs': {
        'avg_price': {'avg': {'field': 'price'}},
        'total_count': {'value_count': {'field': '_id'}}
      }
    }
  }
}
```

Q35: What is the difference between near real-time (NRT) in Elasticsearch and real-time?

Answer: Elasticsearch is described as 'near real-time' — there is a short delay (approximately 1 second) between when you index a document and when it becomes searchable. Why? Elasticsearch writes new documents to an in-memory buffer first. Every 1 second (by default), it 'refreshes' — flushing the buffer to a new Lucene segment, making documents visible to search. This is called the refresh interval. For truly real-time use cases (like logging where every millisecond matters), you can set refresh_interval=1s (default) or even force a refresh manually. For bulk indexing, increasing the refresh interval (e.g., 30s) improves performance significantly.

Section 5: Apache Solr & ES vs Solr Comparison (Q36–Q40)

Apache Solr is Elasticsearch's main competitor — both are built on Apache Lucene. These questions ask you to understand Solr and explain the key differences between the two search engines.

Q36: What is Apache Solr? What are its key features?

Answer: Apache Solr is an open-source search platform built on Apache Lucene. It was created in 2004 at CNET Networks and became an Apache Software Foundation project in 2006 — making it older than Elasticsearch (2010). Key features: Full-text search, faceted search (drill-down filtering), result highlighting, spell correction and autocomplete, rich document parsing (PDF, Word, HTML via Tika), multi-language support, SolrCloud for distributed deployment, support for XML, JSON, and CSV formats for both input and output. Well-known users: Apple, Netflix (historically), IBM, eBay, Walmart.

Q37: What is SolrCloud? How is it different from a standalone Solr instance?

Answer: SolrCloud is Solr's distributed mode — it allows running Solr across multiple servers with automatic sharding, replication, and failover. It uses Apache ZooKeeper for cluster coordination (storing configuration, tracking node health, leader election). Standalone Solr: runs on a single server, manually configured replication, no automatic failover. SolrCloud: distributed across nodes, ZooKeeper manages cluster state, automatic leader election when a node fails, centralised configuration management (config stored in ZooKeeper). Key concept: in SolrCloud, a 'Collection' is the logical index. Each collection is divided into shards, and each shard has replicas for redundancy.

Q38: What is faceted search in Solr/Elasticsearch? Give a real-world example.

Answer: Faceted search is the filtering/drill-down feature you see on e-commerce websites — the left sidebar that shows 'Brand: Nike (150), Adidas (89)' and 'Price: Under ₹1000 (45), ₹1000-₹5000 (230)'. When you click a facet, it filters the results AND shows updated counts for remaining facets. In Solr, facets are configured via facet.field and facet.range parameters. In Elasticsearch, this is done with terms and range bucket aggregations. Both engines handle faceted search extremely well — it's one of the primary use cases for search engines over traditional databases.

Q39: What are the key differences between Elasticsearch and Apache Solr?

Answer: Both Elasticsearch and Solr are excellent search engines built on Lucene. Key differences: Architecture — Elasticsearch is natively distributed, designed for clusters from day one. Solr needed SolrCloud (added later) for distributed operation and requires ZooKeeper as a separate service. Elasticsearch manages its own cluster coordination. Schema — Solr traditionally required a predefined schema (schema.xml), though newer versions support schemaless mode. Elasticsearch is schema-less by default with dynamic mapping. Real-time search — Elasticsearch is faster for real-time indexing and search. Solr can be faster for traditional search on static data. Analytics — Elasticsearch's aggregation framework is more powerful and flexible than Solr's faceting. ELK Stack — Elasticsearch integrates seamlessly with Logstash and Kibana, making it the dominant choice for log analytics.

```
// Quick mental model:
Elasticsearch = simpler to set up + better real-time + ELK ecosystem
Solr = more mature + better for traditional search + more config options
```

Q40: When would you choose Solr over Elasticsearch and vice versa?

Answer: Choose Elasticsearch when: you need real-time search and indexing (logs, monitoring, near-instant search after indexing), you want the ELK Stack (Logstash + Kibana), you need powerful aggregations for analytics, simpler cluster setup (no ZooKeeper dependency), you need geospatial queries, or your team is new to search engines (Elasticsearch has better docs and community).

Choose Solr when: you are working on a traditional, well-established search application where stability matters more than real-time, you need rich document parsing (Solr's Tika integration is excellent for PDF/Word indexing), you need more granular text analysis customisation, or you have an existing Solr deployment that works well. In practice: Elasticsearch has become the dominant choice for new projects. Solr is still widely used in enterprise environments with existing deployments.

Quick Reference: NoSQL Database Comparison

NoSQL Types at a Glance

Type	Example Databases	Data Format	Best For	Weak At
Document Store	MongoDB, CouchDB	JSON/BSON documents	Flexible schemas, product catalogs, user profiles	Complex multi-collection joins
Key-Value Store	Redis, DynamoDB	Key → Value (any)	Caching, sessions, leaderboards, counters	Complex queries on value contents
Wide-Column Store	Cassandra, HBase	Rows with dynamic columns	Time-series, IoT, write-heavy analytics	Complex queries, inconsistent data across rows
Graph Database	Neo4j, Amazon Neptune	Nodes and edges	Social networks, fraud detection, recommendations	Large-scale analytical queries on non-graph data
Search Engine	Elasticsearch, Solr	Indexed JSON documents	Full-text search, log analytics, faceted search	Primary transactional storage

MongoDB vs Redis vs Elasticsearch — Quick Comparison

Feature	MongoDB	Redis	Elasticsearch
Type	Document Store	Key-Value / Data Structure	Search Engine
Storage	Disk (with memory cache)	In-Memory (optionally persisted)	Disk
Primary Use	General-purpose NoSQL DB	Caching, sessions, pub/sub	Full-text search, log analytics
Query Language	MongoDB Query Language	Simple commands (GET, SET)	Elasticsearch DSL (JSON queries)
Schema	Flexible (schema-less)	None (any value per key)	Dynamic mapping (schema-less)
Transactions	ACID (MongoDB 4.0+)	Limited (via MULTI/EXEC)	Not supported (search engine)

Feature	MongoDB	Redis	Elasticsearch
Scaling	Sharding + Replica Sets	Cluster + Sentinel	Shards + Replica Shards
Speed	Fast (disk-based)	Extremely fast (in-memory)	Near real-time (~1 sec delay)
ACID Support	Yes (v4.0+)	Partial (single commands)	No
Best Analogy	A flexible database	A super-fast cache/memory	A search index over documents

Elasticsearch vs Apache Solr — Side by Side

Feature	Elasticsearch	Apache Solr
Year Created	2010	2004 (older)
Built On	Apache Lucene	Apache Lucene
Cluster Coordination	Built-in (no extra tool)	Requires Apache ZooKeeper
Default Schema	Schema-less (dynamic mapping)	Schema required (newer: optional)
Data Format	JSON only	JSON, XML, CSV
Real-time Indexing	Excellent	Good
Aggregations	Very powerful (rich analytics)	Faceting (good, less flexible)
Log Analytics	ELK Stack — industry standard	Less common for logs
Community/Ecosystem	Larger, more active	Mature, enterprise-focused
Setup Complexity	Simpler	More complex (ZooKeeper needed)
Performance	Better for real-time/analytics	Better for traditional static data
Enterprise Features	Elastic Security & Observability	Rich Solr Enterprise features
Choose When	New projects, logs, real-time search	Existing Solr deployments, doc parsing

⚠ Redis is NOT a replacement for MongoDB — Redis is a cache, MongoDB is a database. Use both together.
⚠ Elasticsearch is NOT a primary database — never use it as your only data store (data loss risk). Use it alongside a database.
⚠ MongoDB transactions became ACID in v4.0+ — before that, only single-document operations were atomic.
⚠ In Redis Pub/Sub, messages are lost if no subscribers are listening — use Redis Streams for guaranteed delivery.
⚠ Elasticsearch's 1-second refresh delay is by design — it is NOT real-time, it is NEAR real-time.
⚠ CAP Theorem: MongoDB = CP, Cassandra = AP, Redis = CA (single node) or AP (cluster).